

[BUG] NonDifferentiableError in custom circuit

<https://github.com/PennyLaneAI/pennylane/issues/3632>

ROW 43

[BUG] NonDifferentiableError in custom circuit #3632

New issue



Closed



chstem opened on Jan 13, 2023

...

Expected behavior

As part of a custom VQE implementation, I am calculating some gradients. The provided source code works up to `pennylane==0.26`

Output:

```
-----
pennylane version: 0.26.0
OpenFermion version: 1.5.1
-----
Hamiltonian has 10 qubits
Excitation : [0, 1, 2, 3], Gradient: -0.012964361490340057
Excitation : [0, 1, 2, 5], Gradient: 5.421010862427523e-20
Excitation : [0, 1, 2, 7], Gradient: -1.965116437629976e-19
Excitation : [0, 1, 2, 9], Gradient: 0.03448184841111013
Excitation : [0, 1, 3, 4], Gradient: 4.065758146820651e-20
Excitation : [0, 1, 3, 6], Gradient: -6.776263578034379e-21
Excitation : [0, 1, 3, 8], Gradient: -0.03448184841111031
Excitation : [0, 1, 4, 5], Gradient: -0.023477650505144734
Excitation : [0, 1, 4, 7], Gradient: 0.0
Excitation : [0, 1, 4, 9], Gradient: 0.0
Excitation : [0, 1, 5, 6], Gradient: 0.0
Excitation : [0, 1, 5, 8], Gradient: 5.421010862427569e-20
Excitation : [0, 1, 6, 7], Gradient: -0.02347765050514472
Excitation : [0, 1, 6, 9], Gradient: 0.0
Excitation : [0, 1, 7, 8], Gradient: 2.2361669807513567e-19
Excitation : [0, 1, 8, 9], Gradient: -0.12381871789158702
```



Actual behavior

Using `pennylane==0.28` (same for 0.27), the example outputs:

```
-----
pennylane version: 0.28.0
OpenFermion version: 1.5.1
-----
Hamiltonian has 10 qubits
```



and throws an error (see below).

Additional information

No response

Source code

```
import pennylane as qml
from pennylane import qchem
import openfermion as openfermion
from openfermion.chem import MolecularData
from openfermion.pyscf import run_pyscf
from openfermion.transforms import get_fermion_operator
from openfermion.transforms import jordan_wigner

print('-----')

### packages/libraries info
print('pennylane version:', qml.__version__)
print('OpenFermion version:', openfermion.__version__)

print('-----')

### define input parameters
system = 'system.xyz' #structure file
basis = 'sto-3g'
multiplicity = 1
charge = 0
```



Assignees

soranjh

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



Zoterc

```

basis = sto-3g
multiplicity = 1
charge = 0

active_indices = [1, 2, 3, 4, 5]
occupied_indices = [0]
active_orbitals = 2 * len(active_indices)
active_fermions = 2

# generate a molecule with OpenFermion
molecule = MolecularData(geometry=[('Li', (0.0, 0.0, -0.40)),
                                     ('H', (0.0, 0.0, 1.19))],
                           basis=basis,
                           multiplicity=multiplicity,
                           charge=charge)
molecule = run_pyscf(molecule, run_scf=False)

### generate a Hamiltonian for the VQE approach
fermionic_H = get_fermion_operator(
    molecule.get_molecular_hamiltonian(occupied_indices=occupied_indices,
                                       active_indices=active_indices))
H = jordan_wigner(fermionic_H)

number_qubits = openfermion.utils.count_qubits(H)
print('Hamiltonian has {} qubits'.format(number_qubits))

# translate OpenFermion generated Hamiltonian to PennyLane
H = qchem.import_operator(H, format='openfermion')

# specifying the backend
dev = qml.device('default.qubit', wires=number_qubits)

# generating the hf reference state and all the single and double excitations for UCCSD
def hf_state(active_fermions):
    for i in range(active_fermions):
        qml.PauliX(i)

singles, doubles = qchem.excitations(active_fermions, number_qubits)

# constructing the variational ansatz
def circuit_full(params, wires, excitations):
    hf_state(active_fermions) #global variable

    for i, excitation in enumerate(excitations):
        if len(excitation) == 4:
            qml.DoubleExcitation(params[i], wires=excitation)
        else:
            qml.SingleExcitation(params[i], wires=excitation)

# ground state energy as optimization objective
cost_fn = qml.ExpvalCost(circuit_full, H, dev, optimize=True)
circuit_gradient = qml.grad(cost_fn, argnum=0)

# Compute gradients for all double excitations.

# We initialize the parameter values to zero such that
# the gradients are computed with respect to the Hartree-Fock state.
params = [0.] * len(doubles)
grads = circuit_gradient(params, excitations=doubles)

for i in range(len(doubles)):
    print(f'Excitation : {doubles[i]}, Gradient: {grads[i]}')

```

Tracebacks

```

Traceback (most recent call last):
  File "example.py", line 79, in <module>
    grads = circuit_gradient(params, excitations=doubles)
  File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/pennylane/_grad.py", line 115, in __call__
    grad_value, ans = grad_fn(*args, **kwargs)
  File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/wrap_util.py", line 20, in nary_f
    return unary_operator(unary_f, x, *nary_op_args, **nary_op_kwargs)
  File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/pennylane/_grad.py", line 133, in _grad_with_vjp
    ans = _make_vjp(fun, x)
  File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/core.py", line 10, in make_vjp
    end_value, end_node = trace(start_node, fun, x)
  File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/tracer.py", line 10, in trace
    end_box = fun(start_box)
  File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/wrap_util.py", line 15, in unary_f
    return fun(*subargs, **kwargs)

```

```

File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/core.py", line 20, in more_vjp
    end_value, end_node = trace(start_node, fun, x)
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/tracer.py", line 10, in trace
    end_box = fun(start_box)
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/wrap_util.py", line 15, in unary_f
    return fun(*subargs, **kwargs)
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/pennylane/vqe/vqe.py", line 226, in __call__
    return self.cost_fn(*args, **kwargs)
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/pennylane/vqe/vqe.py", line 217, in cost_fn
    total += sum(r * c_ for r, c_ in zip(res, c))
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/pennylane/vqe/vqe.py", line 217, in <genexpr>
    total += sum(r * c_ for r, c_ in zip(res, c))
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/numpy/numpy_boxes.py", line 27, in _
    def __mul__(self, other): return anp.multiply(self, other)
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/tracer.py", line 46, in f_wrapped
    return new_box(ans, trace, node)
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/autograd/tracer.py", line 118, in new_box
    return box_type_mappings[type(value)](value, trace, node)
File "$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages/pennylane/numpy/tensor.py", line 307, in tens
    raise NonDifferentiableError(
pennylane.numpy.tensor.NonDifferentiableError: -5.731856025596317 is non-differentiable. Set the requires_grad

```

System information

Name: PennyLane
Version: 0.28.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: <https://github.com/XanaduAI/pennylane>
Author:
Author-email:
License: Apache License 2.0
Location: \$HOME/anaconda3/envs/quantum/lib/python3.9/site-packages
Requires: autoray, scipy, numpy, cachetools, appdirs, networkx, autograd, semantic-version, pennylane-lightning
Required-by: PennyLane-Lightning

Platform info: Linux-5.4.0-135-generic-x86_64-with-glibc2.31
Python version: 3.9.7
Numpy version: 1.21.2
Scipy version: 1.8.0

Installed devices:

- default.gaussian (PennyLane-0.28.0)
- default.mixed (PennyLane-0.28.0)
- default.qubit (PennyLane-0.28.0)
- default.qubit.autograd (PennyLane-0.28.0)
- default.qubit.jax (PennyLane-0.28.0)
- default.qubit.tf (PennyLane-0.28.0)
- default.qubit.torch (PennyLane-0.28.0)
- default.qutrit (PennyLane-0.28.0)
- null.qubit (PennyLane-0.28.0)
- lightning.qubit (PennyLane-Lightning-0.28.0)

Existing GitHub issues

☒ I have searched existing GitHub issues to make sure the issue does not already exist.



chstem added **bug** on Jan 13, 2023

soranjh self-assigned this on Jan 13, 2023

soranjh on Jan 13, 2023

Contributor ...

Hi @chstem and thanks for the report. The error is due to using the deprecated `ExpvalCost` class. In `PennyLane-0.28.0` you should get the following warning:

```
UserWarning: ExpvalCost is deprecated, use qml.expval() instead.
```

I modified the code accordingly and it now returns the gradients as expected. Here is the list of the changes made to the original code where the line numbers are for the modified code copied below. Please feel free to let us know if you have any questions. Thanks!

1. Added `@qml.qnode(dev)` to line 60 to create a [QNode](#).
2. Added `return qml.expval(H)` to line 70 to return the expectation value of the Hamiltonian.
3. Removed creating the cost function with `ExpvalCost` in line 74 and used `circuit_full`, as the cost function directly, in `circuit_gradient` in line 75. Your circuit has a `wires` arg which should be defined when you call `circuit_gradient` in line 82.

Modified code:

```
import pennylane as qml
from pennylane import qchem
import openfermion as openfermion
from openfermion.chem import MolecularData
from openfermion.pyscf import run_pyscf
from openfermion.transforms import get_fermion_operator
from openfermion.transforms import jordan_wigner

print('-----')

### packages/libraries info
print('pennylane version:', qml.__version__)
print('OpenFermion version:', openfermion.__version__)

print('-----')

### define input parameters
system = 'system.xyz' #structure file
basis = 'sto-3g'
multiplicity = 1
charge = 0

active_indices = [1, 2, 3, 4, 5]
occupied_indices = [0]
active_orbitals = 2 * len(active_indices)
active_fermions = 2

# generate a molecule with OpenFermion
molecule = MolecularData(geometry=[('Li', (0.0, 0.0, -0.40)),
                                     ('H', (0.0, 0.0, 1.19))],
                           basis=basis,
                           multiplicity=multiplicity,
                           charge=charge)
molecule = run_pyscf(molecule, run_scf=False)

### generate a Hamiltonian for the VQE approach
fermionic_H = get_fermion_operator(
    molecule.get_molecular_hamiltonian(occupied_indices=occupied_indices,
                                       active_indices=active_indices))
H = jordan_wigner(fermionic_H)

number_qubits = openfermion.utils.count_qubits(H)
print('Hamiltonian has {} qubits'.format(number_qubits))

# translate OpenFermion generated Hamiltonian to PennyLane
H = qchem.import_operator(H, format='openfermion')

# specifying the backend
dev = qml.device('default.qubit', wires=number_qubits)

# generating the hf reference state and all the single and double excitations for UCCSD
def hf_state(active_fermions):
    for i in range(active_fermions):
        qml.PauliX(i)

singles, doubles = qchem.excitations(active_fermions, number_qubits)
```

```

# specifying the backend
dev = qml.device('default.qubit', wires=number_qubits)

# generating the hf reference state and all the single and double excitations for UCCSD
def hf_state(active_fermions):
    for i in range(active_fermions):
        qml.PauliX(i)

singles, doubles = qchem.excitations(active_fermions, number_qubits)

# constructing the variational ansatz
@qml.qnode(dev)
def circuit_full(params, wires, excitations):
    hf_state(active_fermions) #global variable

    for i, excitation in enumerate(excitations):
        if len(excitation) == 4:
            qml.DoubleExcitation(params[i], wires=excitation)
        else:
            qml.SingleExcitation(params[i], wires=excitation)

    return qml.expval(H)

# ground state energy as optimization objective
# cost_fn = qml.ExpvalCost(circuit_full, H, dev, optimize=True)
circuit_gradient = qml.grad(circuit_full, argnum=0)

# Compute gradients for all double excitations.

# We initialize the parameter values to zero such that
# the gradients are computed with respect to the Hartree-Fock state.
params = [0.] * len(doubles)
grads = circuit_gradient(params, number_qubits, excitations=doubles)

for i in range(len(doubles)):
    print(f'Excitation : {doubles[i]}, Gradient: {grads[i]}')

```



chstem on Jan 13, 2023

Author ...

Thank you for the fixes. It works very well.



1



chstem closed this as **completed** on Jan 13, 2023